

LED — Liga Acadêmica de Engenharia de Dados · CIn-UFPE

Relatório do Desafio Prático — Etapa 2 · Grupo 17

ELT Analítico — NYC Yellow Taxi (Janeiro/2024)

Pipeline ELT 100% em SQL sobre DuckDB para o aplicativo U-Easy

Equipe

André do Vale Voigt · Antonio Manoel dos Santos Silva

Davi Marcelo Maikel Pedrosa · Mateus Ribeiro de Albuquerque

Julho de 2026

1. Visão geral e arquitetura

Este relatório documenta a solução da equipe para o desafio prático da LED: transformar os registros públicos de corridas dos táxis amarelos de Nova York (janeiro de 2024) em uma base analítica confiável para a equipe fictícia do aplicativo U-Easy, por meio de um processo ELT construído integralmente em SQL sobre o DuckDB.

O processo segue estritamente o padrão ELT: os arquivos de origem (Parquet e CSV) são carregados no banco sem nenhuma edição manual e toda a transformação acontece dentro do ambiente analítico, via SQL. A camada bruta nunca é substituída — cada etapa materializa novas tabelas, preservando a rastreabilidade de ponta a ponta:

```
data/raw (Parquet + CSV)
(1) Extração e carga -> yellow_taxi_raw + taxi_zones_raw      [RAW]
(2) Profiling        -> consultas de investigação
(3) Tratamento       -> yellow_taxi_silver + yellow_taxi_rejected [SILVER + QUARENTENA]
(4) Modelagem         -> fato_viagens + 6 dimensões           [GOLD - Star Schema]
(5) Consumo analítico -> consultas de negócio
```

1.1 Volumetria do pipeline

Camada	Tabela	Linhas	% da fonte
Raw	yellow_taxi_raw	2.964.624	100%
Silver (válidas)	yellow_taxi_silver	2.903.099	97,92%
Quarentena (auditoria)	yellow_taxi_rejected	61.525	2,08%
Gold (fato)	fato_viagens	2.903.099	= Silver

Nenhuma linha é apagada: raw = silver + rejected, e a tabela de quarentena guarda o motivo da rejeição de cada registro (data lineage).

1.2 Por que DuckDB

Além de ser o ambiente recomendado pelo desafio, o DuckDB lê Parquet diretamente com SQL, é colunar e voltado a cargas analíticas (OLAP) — exatamente o perfil do problema: poucas escritas e muitas leituras agregadas sobre cerca de 3 milhões de linhas.

2. Estratégia da equipe

A decisão organizacional mais importante do grupo foi: todos os integrantes construíram individualmente o pipeline até a camada Silver (extração + profiling + tratamento), para que todos aprendessem o processo na prática. Apenas entre cada fase do pipeline o grupo convergiu, comparou as abordagens e consolidou um único tratamento definitivo e um único modelo dimensional, aproveitando a melhor regra de cada um.

Essa estratégia teve dois efeitos deliberados:

- **Aprendizado** — nenhum membro ficou “dono” de uma etapa até a modelagem sem que os outros soubessem reproduzi-la; qualquer integrante consegue explicar qualquer decisão (requisito explícito da avaliação).

- **Qualidade por comparação** — regras concorrentes foram debatidas com evidências do profiling antes de entrar no script definitivo. O script consolidado registra nos comentários a origem de cada regra (ex.: “Regra retirada do script do Mateus”, “Arquitetura de Auditoria do André”).

Os scripts individuais foram mantidos no repositório (src/2_profiling e src/3_tratamento/tratamentos_individuais) como evidência do processo de investigação e das alternativas testadas e descartadas. Na modelagem, o grupo avaliou as propostas e escolheu evoluir o desenho iniciado por André para a versão conjunta final.

3. Reprodutibilidade — ordem de execução

O banco pode ser reconstruído do zero a partir dos arquivos brutos, executando os scripts nesta ordem dentro do DuckDB (./duckdb.exe led_ueasy_taxi.db):

```
.read src/1_extracao/extract_load.sql           -- camada raw
.read src/3_tratamento/tratamento_definitivo/tratamento_gold.sql -- silver + quarentena
.read src/4_modelagem/conjuntas/fact_dim_modeling.sql -- star schema
.read src/5_analise/consultas_analiticas.sql    -- consultas de negócio
```

Os scripts de profiling (src/2_profiling) e de validação (validacao_gold.sql) são opcionais e não alteram o estado final do banco.

Idempotência: todos os scripts usam CREATE OR REPLACE TABLE ou DROP TABLE IF EXISTS antes de criar tabelas. Reexecutar o pipeline quantas vezes for necessário produz exatamente o mesmo resultado — não há INSERT acumulativo nem dependência de estado anterior.

4. Etapa 1 — Extração e carga (Raw)

Tabela	Fonte	Conteúdo
yellow_taxi_raw	yellow_tripdata_2024-01.parquet	2.964.624 corridas (1 linha = 1 viagem)
taxi_zones_raw	taxi_zone_lookup.csv	265 zonas de táxi (lookup LocationID)

- **Carga fiel, sem qualquer transformação.** As tabelas raw são cópia exata dos arquivos — inclusive com nulos, valores negativos e datas impossíveis. Isso preserva a auditabilidade: qualquer regra posterior pode ser conferida contra a fonte original dentro do próprio banco.
- **Leitura direta do Parquet/CSV pelo DuckDB,** sem scripts intermediários em outra linguagem. A inferência de esquema foi conferida no profiling (DESCRIBE / SUMMARIZE) e se mostrou correta, dispensando CAST manual na carga.
- **Alternativa descartada:** converter o Parquet para CSV ou abrir em planilha. Além de proibido pelo desafio, Parquet é um formato binário colunar que não foi feito para edição — toda a exploração foi feita com SQL no profiling.

5. Etapa 2 — Perfilamento dos dados

Cada membro investigou um conjunto de ângulos, com os achados registrados como comentários nos próprios scripts (consultas reproduzíveis, como exige o desafio). Os ângulos foram divididos para evitar retrabalho. As descobertas mais relevantes — e a regra de tratamento que cada uma gerou — estão resumidas abaixo:

Descoberta (evidência no profiling)	Autor(es)	Regra derivada
140.162 linhas com nulos correlacionados: passenger_count, RatecodeID, store_and_fwd_flag, congestion_surcharge e Airport_fee nulos exatamente nas mesmas linhas, vindas inteiramente dos 3 vendedores ativos (vendedor 2: 91.447 · vendedor 1: 48.455 · vendedor 6: 260) — origem comum de ingestão, não falhas independentes	André	Imputação padronizada em vez de exclusão (seção 6)
Essas mesmas 140.162 linhas têm payment_type = 0, código fora do dicionário oficial, mas carregam receita real	Davi	Mapear como membro desconhecido, não apagar
31.465 corridas com 0 passageiros; corridas com 7 a 9 passageiros (vans); absurdos acima de 9	Antonio, André	Preservar o zero, manter 7–9, corrigir > 9
18 registros fora de janeiro/2024 (2002, 2009 e virada de 31/12/2023)	Antonio, André, Mateus	Filtro de escopo temporal
8 viagens com duração negativa (dropoff < pickup) e 681 com duração zero	Antonio, André	Filtro de duração > 0
60.371 corridas com trip_distance <= 0 e máximo absurdo de 312.722 milhas	Antonio, Davi	Filtro de distância > 0
1.024 viagens acima de 100 mph (fisicamente impossível em NYC)	Mateus	Regra de velocidade máxima no quality gate
total_amount diverge da soma dos componentes tarifários em 635.915 linhas (21,4% da base)	Antonio, Mateus	Recalcular total_amount_reconciled
Valores monetários negativos (37.448 em fare_amount, 35.504 em total_amount) concentrados em payment_type 3 e 4 (No charge / Dispute) — estornos, não erros	André	Classificar como Estorno/Disputa em vez de excluir
RatecodeID = 99 significa “nulo” no dicionário — nulo não padronizado	Antonio	Unificar nulo e 99 num único código
Zonas 264/265 = desconhecido/fora de NYC; IDs sem correspondência no lookup	Mateus, Davi	Membros desconhecidos na dim_localizacao
Zero linhas totalmente duplicadas na base bruta	Davi, Mateus	Deduplicação desnecessária — decisão embasada
airport_fee > 0 em zonas que não são aeroportos (115 zonas; East Elmhurst, vizinha ao LaGuardia, lidera com 10.131)	André	Mantido: taxa pode ser legítima no desembarque
Vendedor 7 (Helix) existe no dicionário mas não ocorre nos dados; nenhum código de vendedor fora do domínio	André	dim_vendedor mantém as 4 linhas oficiais do dicionário

Ferramentas exploradas: SUMMARIZE (estatísticas de todas as colunas de uma vez), COUNT(*) FILTER (WHERE ...) para censos de anomalias em uma única varredura, e validação de domínio cruzando os códigos com o dicionário de dados oficial da TLC.

6. Etapa 3 — Tratamento (Silver) e quarentena

6.1 Arquitetura: classificar primeiro, separar depois

O script definitivo cria uma tabela temporária em que cada linha da raw recebe duas colunas de controle: `eh_valido` (booleano) e `motivo_rejeicao` (texto). Só depois as linhas são separadas em `yellow_taxi_silver` (válidas) e `yellow_taxi_rejected` (quarentena).

Por que assim, e não um simples WHERE na criação da silver?

- A regra de validade é definida uma única vez — silver e quarentena são complementos exatos por construção, impossibilitando que uma linha “suma” ou apareça nas duas.
- Cada registro rejeitado carrega o motivo (Fora do escopo, Distância inválida, Duração inválida, Velocidade impossível) — auditoria e data lineage exigidas pelo desafio.
- **Alternativa descartada:** a primeira versão individual usava DELETE/UPDATE sobre uma cópia da tabela (mantida no repositório como registro). Foi descartada porque (a) apagar dados sem trilha viola a auditabilidade, (b) UPDATE/DELETE encadeados dificultam a reexecução idempotente e (c) descartar as 140 mil linhas de telemetria nula jogaria fora receita real.

6.2 Quality gate (o que vai para a quarentena)

Um registro só entra na Silver se todas as condições abaixo valem:

Condição	Justificativa (vinda do profiling)	Rejeitadas
Pickup dentro de jan/2024	Escopo do desafio; registros de 2002/2009/2023 são ruído	18
<code>trip_distance > 0</code>	Corrida sem deslocamento não é viagem analisável	60.371
Duração > 0	Dropoff anterior ou igual ao pickup é fisicamente impossível	112
Velocidade média ≤ 100 mph	Acima disso é falha de GPS/taxímetro, não corrida	1.024
	Total em quarentena (2,08%)	61.525

Detalhe de implementação: a regra de velocidade foi escrita como `trip_distance * 36.0  $\leq$  trip_duration_seconds` (álgebra de `dist/h > 100` mph rearranjada), evitando divisão por zero e comparações de ponto flutuante dentro do filtro.

6.3 Transformações nas linhas válidas (imputação, nunca exclusão silenciosa)

Campo	Regra	Justificativa
<code>passenger_count_clean</code>	<code>NULL -> 1; 0 preservado; > 9 -> 1</code>	Nulo = falha de telemetria (imputa-se o caso mais comum). Zero é evento distinto de nulo (possível cancelamento) e foi mantido. 7–9 são vans legítimas; acima de 9 é erro absurdo. Distinguir ausente $\times$ zero $\times$ incomum $\times$ inválido foi requisito central do desafio.
<code>RatecodeID_clean</code>	<code>COALESCE(RatecodeID, 99)</code>	O dicionário define 99 como Null/unknown — unifica os dois representantes de desconhecido num único código (unknown member da <code>dim_tarifa</code>).
<code>store_and_fwd_flag_clean</code>	<code>COALESCE(flag, 'U')</code>	'U' (Unknown) mantém o domínio Y/N/U explícito em vez de nulo.

Campo	Regra	Justificativa
congestion_surcharge, Airport_fee	COALESCE(..., 0.0)	Taxa ausente = taxa não cobrada; zero mantém as somas financeiras corretas.
tipo_transacao	'Estorno/Disputa' se payment_type IN (3,4) ou valores negativos	Valores negativos não são erro — concentram-se em No charge/Dispute. Excluí-los inflaria o faturamento; a classificação preserva o sinal contábil.
total_amount_reconciled	Soma recalculada dos componentes tarifários	O profiling encontrou divergências entre total_amount e a soma dos componentes; o recálculo garante consistência interna.
trip_duration_seconds, average_speed_mph	Colunas derivadas materializadas	Feature engineering antecipado: evita repetir a mesma expressão de data em todas as consultas.

6.4 Alternativas descartadas nesta etapa

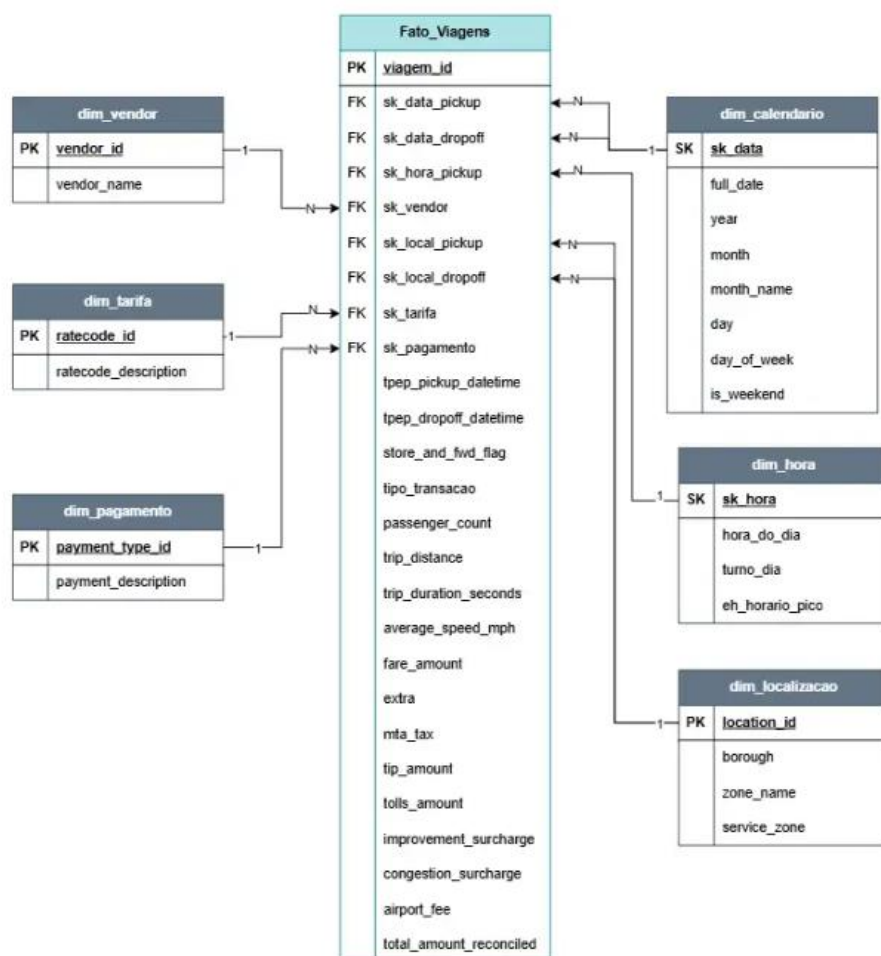
- **Excluir as 140.162 linhas de telemetria nula** — descartada com evidência: essas linhas têm receita real (payment_type 0 = Flex Fare). A imputação preserva ~4,7% da base.
- **Excluir corridas com passenger_count = 0** — descartada: o zero é informação (cancelamento/no-show), não ausência.
- **Corrigir total_amount apenas onde divergisse** — descartada em favor do recálculo uniforme, mais simples de auditar e idempotente.
- **Filtrar viagens muito curtas (< 30–60s)** — investigada no profiling (26.603 viagens <= 30s), mas o grupo rejeitou apenas o fisicamente impossível. Viagem curta é incomum, não inválida.

7. Etapa 4 — Modelagem dimensional (Star Schema)

7.1 Grão da tabela fato

Uma linha de fato_viagens = uma corrida de táxi individual, validada pelo quality gate, com pickup em janeiro de 2024. O grão da fonte (1 linha = 1 viagem) foi mantido — é o menor nível de detalhe possível, permitindo qualquer rollup (hora, turno, dia, semana, mês) sem pré-agregação.

7.2 O modelo



A fato central conecta-se a seis dimensões por chaves inteiras. As dimensões de calendário e localização são role-playing (usadas duas vezes cada: pickup/dropoff e origem/destino):

Dimensão	Linhas	Origem	Papel analítico
dim_vendor	4	Dicionário oficial (VALUES)	Market share por provedor de táxi
dim_tarifa	7	Dicionário oficial; 99 = unknown member	Análise por tipo de tarifa (JFK, negociada...)
dim_pagamento	7	Dicionário oficial; 0 = Flex Fare	Gorjeta por forma de pagamento
dim_calendario	32	Derivada das datas da Silver (chave YYYYMMDD)	Dia da semana, fim de semana — role-playing (pickup/dropoff)
dim_hora	24	RANGE(0,24) — todas as horas	Turno do dia e flag de horário de pico
dim_localizacao	265+	taxi_zones_raw + unknown members (264/265)	Zona, borough — role-playing (origem/destino)

Na fato ficam as chaves das dimensões, duas dimensões degeneradas (tipo_transacao e store_and_fwd_flag), as medidas operacionais (passageiros, distância, duração, velocidade média) e as medidas financeiras (os 9 componentes tarifários e o total reconciliado).

7.3 Decisões de modelagem e alternativas descartadas

- **Chaves naturais nas dimensões, surrogate apenas na fato.** Os códigos da fonte são estáveis, pequenos e definidos por dicionário oficial — uma surrogate key só adicionaria um JOIN de tradução sem ganho (não há histórico/SCD num recorte de um mês). A fato recebeu viagem_id gerado por ROW_NUMBER() sobre uma ordenação determinística de 8 colunas: a fonte não tem chave natural, e a ordenação estável garante os mesmos IDs em reexecuções.
- **Tempo separado em duas dimensões (data e hora) com chave inteira.** Alternativa descartada: uma única dimensão datetime — teria milhões de membros. A separação dá 32 + 24 linhas e cobre as perguntas do negócio. A chave YYYYMMDD inteira é o padrão de data warehouse: legível, ordenável e barata para JOIN.
- **dim_hora gerada por RANGE(0,24), não a partir dos dados** — garante as 24 horas mesmo sem corridas, e materializa turno_dia e eh_horario_pico (7–9h / 16–19h) em vez de repetir o CASE em cada consulta.
- **Unknown members na dim_localizacao.** As zonas 264 (desconhecida) e 265 (fora de NYC) são garantidas via UNION ALL ... WHERE NOT EXISTS, e a fato usa COALESCE(LocationID, 264) — nenhuma corrida é descartada por origem/destino desconhecido.
- **Star schema, não snowflake.** Normalizar borough/service_zone em tabelas próprias (outrigger) economizaria bytes irrelevantes e custaria um JOIN a mais em toda consulta geográfica — contra o princípio OLAP de favorecer leitura.
- **Dimensões degeneradas na fato.** tipo_transacao e store_and_fwd_flag são atributos de 2–3 valores sem outro contexto descritivo — criar uma dimensão para cada seria fragmentação excessiva.
- **LEFT JOIN (nunca INNER) na montagem da fato** — um código sem correspondência jamais elimina uma corrida silenciosamente; o caso é resolvido por unknown member e conferido pelas validações de órfãos.

8. Etapa 5 — Consultas analíticas

Todas as consultas usam exclusivamente o modelo final (fato_viagens + dimensões) — nunca a raw. Cada consulta está documentada no script com o par Decisão/Motivo, conectando o SQL à pergunta de negócio da U-Easy:

Perspectiva	Perguntas respondidas	Uso pelo negócio
Temporal	Corridas e faturamento por turno e horário de pico; volume e duração por dia da semana	Tarifa dinâmica (surge pricing), alertas de escassez de motoristas, alocação de frota
Geográfica	Top 10 rotas origem-destino; volume e faturamento por borough	Corredores de mobilidade, pontos fixos de embarque, expansão territorial
Financeira	Decomposição do faturamento; gorjeta por forma de pagamento; impacto de estornos	Transparência para a diretoria; cartão incentiva gorjeta; apoio ao time de risco
Operacional	Velocidade e duração em pico × fora de pico; distribuição de ocupação	Calibração do ETA do aplicativo; dimensionamento de cancelamentos
Estratégica	Market share e faturamento por vendedor de tecnologia	Negociações B2B com fornecedores de taxímetro

As consultas usam apenas agregações e JOINS sobre chaves das dimensões — sem lógica de tratamento embutida (turno, pico, fim de semana e zonas já vêm prontos das dimensões), exatamente o formato que alimentaria um dashboard para usuários não-técnicos.

9. Validação, auditoria e reconciliação

O pipeline comprova que produziu dados confiáveis em três pontos:

- **Auditoria da Silver:** escopo temporal conferido (mínima 2024-01-01 00:00:00, máxima 2024-01-31 23:59:55); zero nulos remanescentes nos campos tratados; 30.569 zeros de passageiros preservados e 37 vans legítimas (7–9 passageiros) mantidas; tolerância zero para ruído físico (as três regras do quality gate retornam 0 ocorrências na Silver); distribuição da quarentena por motivo, sem rejeitado sem motivo; fechamento financeiro por categoria: Regular 2.835.727 corridas / \$ 78.930.412,32 × Estorno/Disputa 67.372 corridas / \$ 20.939,03.
- **Reconciliação de contagens:** raw – silver – rejected = 0 (nenhuma linha perdida ou duplicada no caminho); contra-prova de que nenhum registro válido ficou preso na quarentena.
- **Validações do modelo dimensional:** unicidade das chaves de todas as dimensões; fato com a mesma contagem da Silver e viagem_id 100% único; zero chaves nulas e zero órfãos — todo FK da fato encontra sua dimensão (integridade referencial testada por LEFT JOIN).

10. Papel de cada membro da equipe

A tabela reflete a estratégia da seção 2: todos fizeram profiling e tratamento; as atribuições destacam a contribuição distintiva de cada um, evidenciada nos scripts do repositório.

Membro	Contribuições principais
André do Vale Voigt	Profiling de nulos correlacionados (descoberta de que as 140 mil linhas de telemetria vêm dos mesmos vendedores — origem comum de ingestão), datas, valores negativos, airport_fee e domínio de vendedores; arquitetura de classificação e quarentena (eh_valido + motivo_rejeicao); classificação Estorno/Disputa; métricas derivadas (duração/velocidade); modelagem dimensional (grão, estratégia de chaves, dimensões + fato com validações) que serviu de base à versão final; script de reconciliação raw × clean × rejected.
Antonio Manoel dos Santos Silva	Extração e carga da raw; profiling de passageiros (insight das vans de 7–9), datetimes, distâncias, RatecodeID e conciliação total_amount × componentes; primeira abordagem de tratamento (evolução documentada de DELETE para flag de exclusão); consolidação da camada gold.
Davi Marcelo Maikel Pedrosa	Profiling de distância, forma de pagamento e zonas geográficas; descoberta de que payment_type = 0 coincide com as linhas de telemetria nula (reforçou a decisão de imputar em vez de excluir); verificação de duplicatas exatas.
Mateus Ribeiro de Albuquerque	Profiling estrutural mais amplo (tabela consolidada de inconsistências: domínios do dicionário, órfãos de FK, velocidade impossível, divergência matemática de 21,4%, mapa de nulos, cardinalidades); tabela de transformações e regras de “salvação de receita” que ancoraram o tratamento definitivo da Silver; filtros físicos; auditorias da Silver.

11. Uso de LLM

Em conformidade com a seção 05 do enunciado (LLM como tutora, não executora):

- Toda a lógica de SQL — extração, profiling, regras de tratamento, modelagem e consultas — foi escrita, testada e é explicável pelos integrantes; os scripts individuais mantidos no repositório documentam o processo real de investigação e as iterações de cada membro.
- Este relatório foi redigido com apoio de LLM a partir dos scripts e decisões já existentes no repositório, atuando como revisão e estruturação textual — o conteúdo técnico e as decisões descritas são as que estão materializadas nos arquivos SQL do grupo.